

CS & IT ENGINEERING



Data Structure & Programming

Stack and Queues

Lecture No.- 05



By- Satya sir

Recap of Previous Lecture



- Queue

- Nature

- operations

- Types of Queues

- Simple Q [SQ] } FIFO

- Circular Q [CQ] }

- Double Ended Q [DQ] { Same Ends LIFO
diff Ends FIFO

- Priority Q [PQ]



Topics to be Covered



- Simple Queue
 - Circular Queue
 - Double Ended Queue
 - Priority Queue
 - Stack Implementation
- } operations



Topic : Types Of Queues, Operations



Simple Queue : Enqueue() $\rightarrow$ rear End, Dequeue() $\rightarrow$ front End.

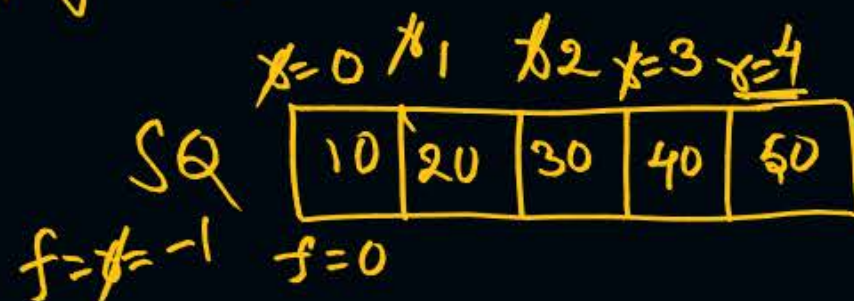
- Let SQ is implemented Using array.

SQ be Simple Queue of size = 5, Initially Empty. $f = r = -1$

Void Enqueue(Value, SQ)

```
{
    if(  $r == \text{SIZE} - 1$  )
        Printf("Queue is Full");
    else
    {
         $r = r + 1$ ;
         $\text{SQ}[r] = \text{Value}$ ;
        if(  $r == 0$  ) (or) if(  $f == -1$  )
             $f = 0$ 
    }
}
```

Time
Complexity
 $O(1)$



Enqueue(10, SQ) $\Rightarrow r = r + 1 \Rightarrow r = 0$ $\text{SQ}[0] = 10$, $f = 0$

Enqueue(20, SQ) $\Rightarrow r = 1$, $\text{SQ}[1] = 20$; $r = 1$, $f = 0$

Enqueue(30, SQ) $\Rightarrow r = 2$, $\text{SQ}[2] = 30$ $r = 2$, $f = 0$

Enqueue(40, SQ) $\Rightarrow r = 3$, $\text{SQ}[3] = 40$ $r = 3$, $f = 0$

Enqueue(50, SQ) $\Rightarrow r = 4$, $\text{SQ}[4] = 50$, $r = 4$, $f = 0$

Enqueue(60, SQ) $\Rightarrow r == \text{SIZE} - 1$ Queue is Full.
True



Topic : Types Of Queues, Operations



f=0, r=4

qp

void Dequeue (SQ)

```
{
    if (f == -1)
        Printf("Queue is Empty");
    else
    {
        Printf("Deleted value is %d", SQ[f]);
        if (f == r)
        {
            f = -1;
            r = -1;
        }
        else f = f + 1;
    }
}
```

0 == -1 F Dequeue (SQ); Deleted value is 10 f=1, r=4
1 == -1 F Dequeue (SQ); " " 20 f=2, r=4
2 == -1 F Dequeue (SQ); " " 30 f=3, r=4
3 == -1 F Dequeue (SQ); " " 40 f=4, r=4
4 == -1 F Dequeue (SQ); " " 50 f=-1, r=-1 True
-1 == -1 True Dequeue (SQ) f=-1 True Queue is Empty.

Time Complexity : $O(1)$



Topic : Types Of Queues, Operations



Circular Queue

Drawback of Simple Queue

Let $SQ[5]$ be initially empty. $f = -1, r = -1$

$f = 0$ $f = 1$ 2 3 $f = 4$
 $r = 4$



Enqueue(10, SQ) $r = 0, f = 0$
Enqueue(20, SQ) $r = 1, f = 0$
Enqueue(30, SQ) $r = 2, f = 0$
Enqueue(40, SQ) $r = 3, f = 0$
Enqueue(50, SQ) $r = 4, f = 0$

Dequeue(SQ) $f = 1, r = 4$
Enqueue(60, SQ) $r == SIZE - 1 \Rightarrow 4 == 4$ True
Queue is Full
Dequeue(SQ) $f = 2, r = 4$
Enqueue(60, SQ) $r == SIZE$
Queue is ...

Dequeue(SQ), Enqueue(60, SQ)
Q is Full
Dequeue(SQ), Enqueue(60, SQ)
Q is Full
Dequeue(SQ), Enqueue(60, SQ)
 $f = r = -1$, 60 insert at
index 0.

In Simple Queue, Once r reaches max index,
Until Complete Queue is Emptied, No Element Insertion
will be possible. Wastage / Poor Utilization of memory space.



Topic : Types Of Queues, Operations



Circular Queue

In CQ, front and rear need to be updated in cyclic (or) Circular manner.

$$r = (r + 1) \% \text{SIZE}$$

$$f = (f + 1) \% \text{SIZE}$$

Enqueue(10, CQ) $r=0, f=0$
Enqueue(20, CQ) $r=1, f=0$
Enqueue(30, CQ) $r=2, f=0$
Enqueue(40, CQ) $r=3, f=0$
Enqueue(50, CQ) $r=4, f=0$
Dequeue(CQ) $r=4, f=1$
Enqueue(60, CQ) $r=0, f=1$



In Circular Queue :

Full: $f == (r + 1) \% \text{SIZE}$

Empty: $f == r$ or $f == -1$

Dequeue(CQ) $r=0, f=2$
Dequeue(CQ) $r=0, f=3$
Dequeue(CQ) $r=0, f=4$
Dequeue(CQ) $r=0, f=0$
Dequeue(CQ) if $f == r \Rightarrow f = r - 1$



Topic : Types Of Queues, Operations



Double Ended Queue (DEQUEUE)

In DQ, Insertion and Deletion can be done from both Ends / Either Ends.

Let Initially, DQ be Empty. $f = -1, r = -1, SIZE = 7$

Enqueue_rear(10, DQ) $f = 0, r = 0, f = 0$

Enqueue_front(20, DQ) $r = 0, f = 6$

Enqueue_rear(30, DQ) $r = (r+1) \% SIZE, r = 1, f = 6$

Enqueue_front(40, DQ) $f = f-1, r = 1, f = 5$

Enqueue_front(50, DQ) $f = f-1, r = 1, f = 4$

Enqueue_rear(60, DQ) $r = (r+1) \% SIZE, r = 2, f = 4$

Enqueue_rear(70, DQ) $\rightarrow r = 3, f = 4$

(or) Enqueue_front(70, DQ) $\rightarrow r = 2, f = 3$ ✓



0	1	2	3	4	5	6
10	20	30	40	50	60	70
r, f	r	r	f	r	f	$f = 6$

 DQ FULL: $f == (r+1) \% SIZE$
 DQ Empty: $f == r$ (or) $f == -1$

Dequeue(front) : $f = (f+1) \% SIZE, f = 4, r = 2$

Dequeue(rear) : $r = r-1, f = 4, r = 1$

Dequeue(front) : $f = 5, r = 1$

Dequeue(rear) : $f = 5, r = 0$

Dequeue(rear) : if $r == 0, r = SIZE-1, r = 6, f = 5$

Dequeue(rear) : $r = 5, f = 5$

Dequeue(front) (or) Dequeue(rear) : Delete, if $f == r, f = -1, r = -1$



Topic : Types Of Queues, Operations



Priority Queue $\Rightarrow$ It is Used in Binary Heap Implementation and Heap Sort algo.

- Deletion of Elements happen based on Priority.

Insertion Sequence: 10, 20, 30, 40, 50

Deletion Sequence: 40, 20, 10, 50, 30

	$x=0, f=0$	$x=1$	$x=2$	$x=3$	$x=4$
Value	10	20	30	40	50
Priority	3	4	1	5	2
	$f=0$	$f=1$		$f=3$	

Enqueue (value, Priority, PQ)

$x=0, f=0$

Enqueue (20, 4, PQ)

$x=1, f=1$

Enqueue (30, 1, PQ)

$x=2, f=1$

Enqueue (40, 5, PQ)

$x=3, f=3$

Enqueue (50, 2, PQ)

$x=4, f=3$

Dequeue (PQ): 40 deleted, $f=1$, $x=3$

Dequeue (PQ): 20 deleted, $f=0$, $x=1$

Dequeue (PQ): 10 " $f=4$, $x=0$

Dequeue (PQ): 50 " $f=2$, $x=4$

Dequeue (PQ): 30 " $f=-1, x=-1$



Topic : Stack Implementation



Stack Implementation

Stack may be implemented Using

- 1) An Array ✓
- 2) Linked List
- 3) Queue

Stack

Linked List Implementation: Push : Insertion at beginning $O(1)$
Pop : Deletion of first node $O(1)$

Push Sequence : 10, 20, 30
Pop Sequence : 30, 20, 10



Queue Implementation

Queue may be Implemented Using

- 1) An Array ✓
- 2) Linked List
- 3) Stack

Queue

Enqueue : Insertion at End $O(n)$ / Insertion at Beginning $O(1)$
Dequeue : Deletion of first node $O(1)$ / Delete last node $O(n)$

Insertion Sequence : 10, 20, 30





2 mins Summary



- Simple Queue
 - Circular Queue
 - DEQUEUE
 - Priority Queue
- } Operations
- Stack / Queue Implementation Using Linked Lists

To be Contd... 😊



THANK - YOU